Laboratory Work 2 - REST API and Message Queues

Author: Musagaliev Rahat
Course: Distributed & Cloud Computing
Git: https://github.com/bl00dtearz/lab2_cloud

Queue variant: C-2 (file-based queue, no AWS).

Answers to the reflection questions

1. How does REST fundamentally differ from the socket protocol of Laboratory Work 1? What does it give that every request is self-contained?

In Lab 1 we made our own text protocol (ECHO / TIME / QUIT) on top of a socket. The connection stayed open until QUIT, and the server worked with one client connection at a time. REST uses the standard HTTP protocol and JSON. A request has a URL, a method (GET, POST), a body and the answer has a status code. Any client can use it: curl, a browser or another program, without a special client.

Because every request is self-contained, the server does not remember earlier requests. It does not need an open connection. So it is easy to run several copies of the service behind a load balancer: any copy can answer any request. It is also easier to test (one request, one answer) and to restart the service without breaking a conversation.

2. Why does the job-submission endpoint return code 202 and not 200?

Code 200 means that the work is finished and the answer is ready. When the client sends POST /jobs, the API only puts the job into the queue. The job is not done yet: the worker will process it later (about 2 seconds). Code 202 Accepted says exactly this: "accepted for processing, the result will come later". The client gets the job_id and asks for the result with GET /jobs/<job_id>.

3. Why is a queue needed if the job could be done directly inside the request? When does it matter?

If the job is done inside the request, the client must wait until it finishes. A long job can block the server and can cause a timeout. With a queue, the API answers fast (202 and job_id) and the worker does the slow work in a separate program at its own speed. If the worker is stopped, the jobs are not lost: they wait in the queue (see the screenshot with the file in queue/).

It matters when the work is slow (reports, video, e-mails, file processing), when many requests come at the same time (the queue smooths the peaks), and when the worker can fail or must be scaled separately from the API.

4. What happens if the worker crashes after receiving a message but before deleting it from the queue?

In my worker the job file is deleted only after the result is saved to results/<job_id>.json. So if the worker crashes in the middle, the file stays in queue/. After restart the worker finds it again and processes it again. This is "at least once" delivery. The result is written to a temporary file and then renamed, so a half-written result is never visible. The processing (upper case) gives the same result every time, so doing it twice is safe. In Amazon SQS the idea is the same: the message becomes invisible for the visibility timeout, and if it is not deleted, it returns to the queue.

Screenshots



Fig. 1. Successful requests: POST /echo returns the reply, GET /time returns the current UTC time (the server log on the left shows 200).

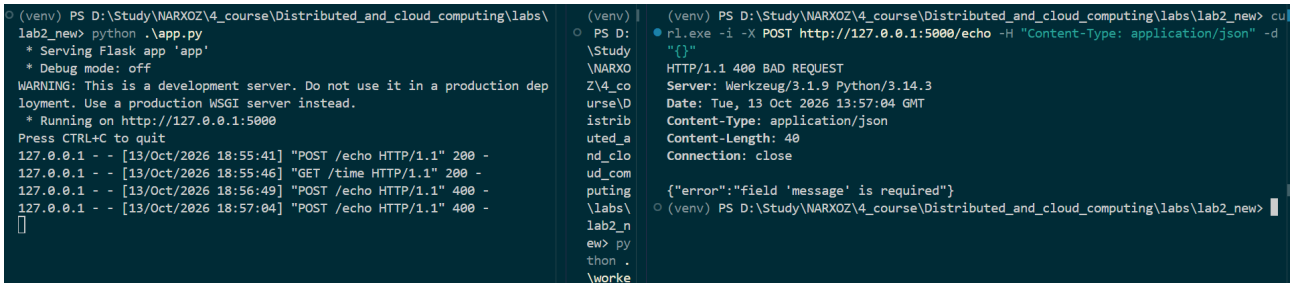


Fig. 2. Invalid data: POST /echo with an empty JSON {} returns 400 Bad Request and a JSON error.

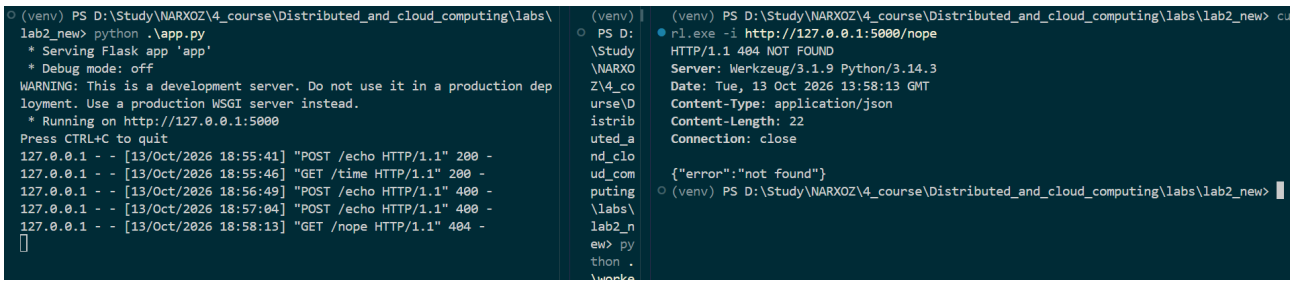


Fig. 3. Unknown path /nope returns 404 Not Found and a JSON error.



Fig. 4. Job submission: POST /jobs returns the job_id and status queued. The server log on the left shows code 202 for this request.



Fig. 5. Job status after the worker has processed it: done, result "HELLO WORLD".

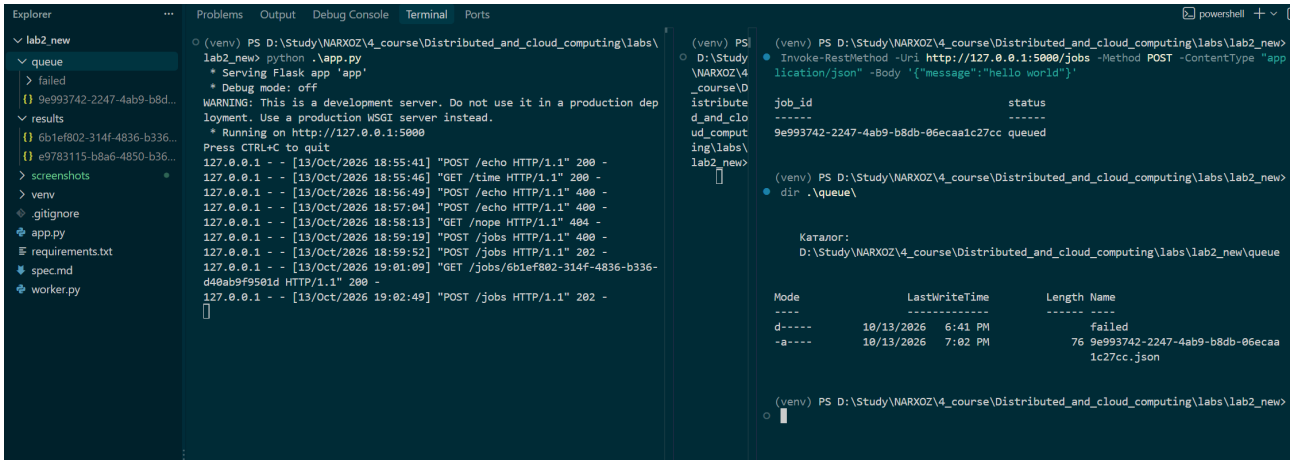


Fig. 6. The queue: the worker is stopped, and dir queue shows the job file waiting for processing.